[Ch.4 분류 part 1]

분류(classification)의 개요: 지도학습의 대표 유형 분류

➔ 학습데이터(데이터 피처나 레이블 값)으로 학습해 미지 레이블 값 예측

• 지도학습 : 레이블(Label), 즉 **정답이 있는 데이터**가 주어진 상태에서 학습하는 머신러닝 방식

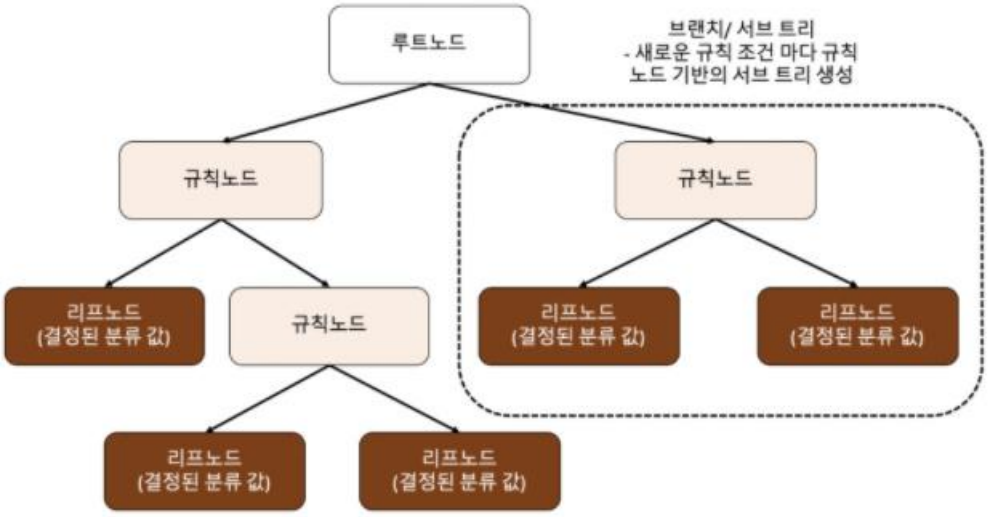
분류를 구현해주는 머신러닝 알고리즘 : 로지스틱 회귀, 나이브 베이즈, 결정 트리, 최소 근접, 앙상블 등이 있다

- 앙상블 방법은 정형 데이터를 예측함에 있고 배깅 방식 [예] 랜덤 포레스트], 부스팅 방식[예] 그래디언트 부스팅], 앙상블의 기본 알고리즘이 결정트리임

결정 트리 = 가지 뻗기 기법

규칙을 학습 & 패턴 발견 → 트리(Tree) 기반의 분류 규칙 생성

if, else 규칙 기반 → 규칙을 만들어내는 '기준 '



루트 노드	규칙 노드	리프 노드	서브(브랜치) 트리
- 근원 - 시작점	- 규칙 조건시작 - 여기서 분류됨	- 결정된 값 - 클래스 값	- 규칙노드에 위배된 변수가 있으면 생성 - 새 규칙 조건마다 0

(+) 규칙이 많음 ->복잡함-> 과적합일 수 있으므로, **가장 적은 노드로 최대한 균일하게 나누어야함**

그렇다면 균일함이란? = 같은 데이터가 많음=서브트리가 거의 없음

> 균일도의 측정법 두가지

- **정보 이득지수 = 1- (엔트로피)로 높을수록** 좋고, 결정트리는 정보 이득 지수로

분할함 [정보이득 = 엔트로피(=혼잡도) 낮으면 좋음]

- **지니계수 = 낮으면** 좋은 것 (decisiontreeclassifier은 엔트로피로 분리하는 것이 default)

파라미터명	설명
min_samples_split	샘플이 최소한 몇개 이상이어야 split(하위(잎)노드로 분리)할것인가/클수록 과적합방지, 작을수록 정확하게 분리되어 과적합
min_samples_leaf	(잎) 노드가 되려면 가지고 있어야 할 최소 샘플의 수/ 클수록 과대적합방지, 작을수록 과적합/ min_samples_split을 만족해도 min_sample_leaf을 만족하지 않으면 leaf노드가 되지 못한다.
max_depth	얼마나 깊게 트리를 만들것인가/None이면 최대한 깊게(불순도가 0이 될때까지)/클수록 과적합,작을수록 과적합방지
max_leaf_nodes	최대 몇개 잎 노드가 만들어 질때 까지 split(하위(잎)노드로 분리)할것인가
max_feature	최적의 분할을 위해 고려할 최대 피처개수

- 저번 시간에 배운 f1 score을 통해 결정트리를 평가함

선형 모델에서는 특성, 타겟 관계 확인을 위해 회귀계수 확인하나,

결정트리에서는 **특성중요도**(항상 양, 특성이 얼마나 자주 분기에 사용되는지)를 확인함

 # 이전 붓꽃 dataset에 결정 트리 적용해 서브 트리 작성한 시각화자료

순서 1. DecisionTree Classifier 생성

2. 붓꽃 데이터 로딩 후 학습용, 테스트용 데이터셋 분리

3. 학습 진행 및 export_graphviz()결과로 지정된 파일 생성

4. 위의 tree.dot 파일을 Graphviz 읽어서 주피터 노트북에서 시각화

#하이퍼 파라미터

Max_depth : 최대 depth를 고정시켜 간단한 트리 구조형성

Min_samples_split : 최소 n개의 sample이 있어야 자식 노드를 만들게 함

min_samples_leaf : 리프 노드(=자식 노드 없는 노드)가 될 수 있는 최소한의 sample

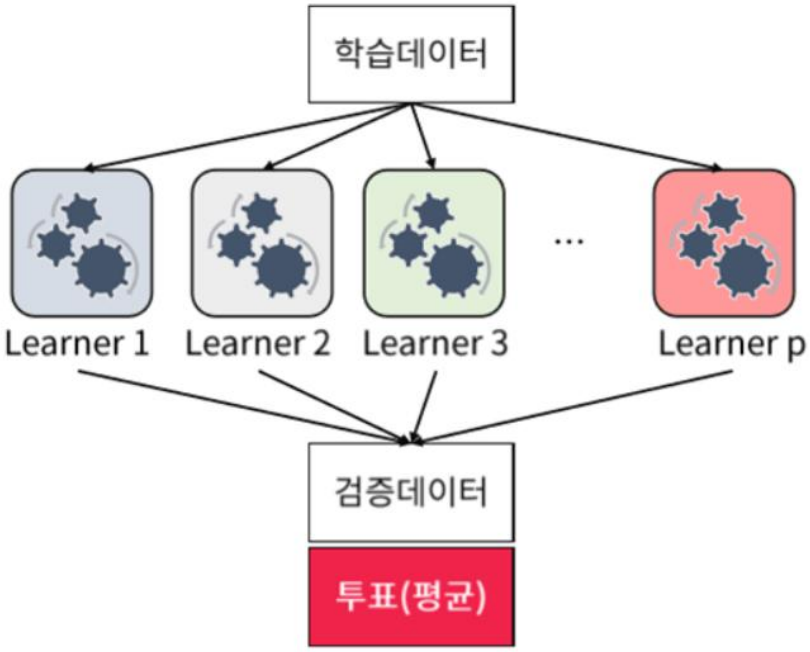
의 수

 # 결정 트리 과적합

make_classification() 함수로 우선 test용 데이터 제작(사이킷런에서!)

앙상블 학습 : 여러 개의 분류기(Classifier)를 생성하고 그 예측을 결합

1. • 정형 데이터분류, 회귀에서 매우 강한 성능을 보임[정확, 안정성, 신뢰성]



1. Voting과 Bagging: 여러 개의 분류기가 투표를 통해 최종 예측 결과를 결정하는 방식

Voting은 서로 다른 알고리즘을 가진 분류기 결합

Bagging은 각각의 분류기가 모두 같은 유형의 알고리즘 기반이지만, 데이터 샘플링을 서로 다르게 가져가면서 학습을 수행해 **Voting을 수행하는 것**

- **Bagging: bootstrap aggregating의 줄임말**
- **1. Bootstrapping 분할 방식 – 데이터 중첩 허용**
- **크기만큼 bootstrap 실행(복원 추출 개념)**
- **각 분류기 k개만큼 데이터를 샘플링 -> 개별적인 예측을 보팅(결과 값 평균)**
- **OOB error(Out-of-Bag error): 학습데이터에서 미 추출된 데이터에 대해 각각의 분류기가 예측하고 Error율 계산해서 평균 냄**

-> 학습데이터 내에서 미 추출된 데이터를 검증 데이터로 써서 검증데이터에 대한 성능 지표를 계산할 수 있는 장점이 있음

위에서 말했던 결정 트리의 과적합 문제를 해결하기 위해 평균 사용해 편향을 유지하고, 분산을 줄여 데이터 노이즈를 지우며 bias-variance trade-off효과

2. **Boosting: 여러 개의 분류기가 순차적으로 학습을 수행하되, 앞에서 학습한 분류기가 예측이 틀린 데이터에 대해서 올바르게 예측할 수 있도록 다음 분류기에게 가중치(weight)를 부여하면서 학습과 예측 진행**

- **예측 성능이 뛰어나 앙상블 학습을 주도**

3. **Stacking**: 여러 가지 다른 모델의 예측 결과값을 다시 학습 데이터로 만들어서 다른 모델(메타 모델)로 재학습시켜 결과를 예측

2. 보팅 유형 – 하드 보팅(Hard Voting)과 소프트 보팅(Soft Voting)

1. **하드 보팅**: 예측한 결과값들 중 다수의 분류기가 결정한 예측값을 최종 보팅 결과값으로 선정 (다수결 원칙과 비슷)!
2. **소프트 보팅**: 분류기들의 레이블 값 결정 확률을 모두 더하고 이를 평균해서 이들 중 확률이 가장 높은 레이블 값을 최종 보팅 결과값으로 선정 (일반적인 보팅 방법)

랜덤 포레스트 : bagging의 대표 알고리즘! – 빠른 속도와 높은 예측력

여러 개의 결정 트리 분류기가 <u>전체 데이터에서 Bagging 방식으로 각자의 데이터를 샘플링해 개별적으로 학습을 수행</u>한 뒤 <u>보팅</u>을 통해 예측 결정

Bagging model(여러 트리들)의 분산은 각각 트리들의 분산과 그들의 공분산으로 이루어져있음

$$Var(X+Y) = Var(X) + Var(Y) + \underline{2Cov(X,Y)}$$

↓

전체데이터에서 복원 추출하였으나, 각각의 트리들은 중복되는 데이터를 다수 가지고 있기 때문에 독립이라는 보장이없음

↓

$Cov(X,Y) = 0$ 이라는 조건을 만족하지 못함 (비슷한 tree가 만들어질 확률이 높음)

↓

Tree가 증가함에 따라 오히려 모델 전체의 분산이 증가 할 수도 있음

=> 단점

↓

각 Tree간 공분산을 줄일 수 있는 방법이 필요함

공분산을 최소화 (0가능)하고자 함

➔ 기본 bagging 과 다르게 데이터 뿐 아니라 변수도 랜덤 추출! -> 성능 좋음

랜덤 포레스트 하이퍼 파라미터

하이퍼 파라미터 종류

n_estimator: 결정 트리의 개수 지정. Default = 10
max_features: 결정 트리에 사용된 max_feature파라미터와 동일. 기본형: 'auto'(=='sqrt(제곱근)') =>트리분할 피처 참조 시 전체 피처X, sqrt(전체 피처개수)만큼 참조
max_depth, min_samples_leat, min_samples <=결정 트리에서 과적합 개선하는 파라미터가 똑같이 적용될 수 0